

# How to Choose Activation Functions

The choice of activation function is a key part of designing a neural network. The rules for choosing are different for the **output layer** versus the **hidden layers**.

## 1. Choosing the Output Layer Activation Function

The choice for the output layer is determined by the **type of problem** and the **range of your target variable  $y$** .

| Problem Type & $y$ Value                      | Recommended Activation       | Why?                                                                              |
|-----------------------------------------------|------------------------------|-----------------------------------------------------------------------------------|
| Binary Classification ( $y$ is 0 or 1)        | Sigmoid                      | Outputs a probability between 0 and 1, just like logistic regression.             |
| Regression ( $y$ can be positive or negative) | Linear ( $g(z) = z$ )        | The output ( $a$ ) needs to be able to take on any value, positive or negative.   |
| Regression ( $y$ is non-negative, $\geq 0$ )  | ReLU ( $g(z) = \max(0, z)$ ) | The output ( $a$ ) is guaranteed to be 0 or positive, matching the range of $y$ . |

This is the most common and recommended approach for setting the output layer.

## 2. Choosing the Hidden Layer Activation Function

For all hidden layers, the modern default and most common choice is **ReLU**.

- **Recommendation:** Use **ReLU** for all hidden layers.

### Why ReLU is Preferred Over Sigmoid (for Hidden Layers)

Historically, sigmoid functions were used in hidden layers, but this practice is now rare. ReLU is preferred for two main reasons:

1. **Faster Computation:** The  $\max(0, z)$  operation is computationally simpler and faster than the sigmoid function (which involves an exponent and division).
2. **Faster Training (Gradient Descent):** This is the more important reason.
  - The **sigmoid function** "goes flat" (saturates) in two regions: when  $z$  is very large or very small.

- These **flat spots** have a gradient (slope) of almost zero.
  - A zero-gradient **stops gradient descent from learning** or slows it down significantly.
  - The **ReLU function** is only flat in one region (when  $z$  is negative). It does not saturate for positive values.
  - This means that, on average, learning (gradient descent) is much faster with ReLU.
- 

## Implementation in TensorFlow

You specify the activation function as a string in the Dense layer.

- **Example:** A network for binary classification (like the one above) would be built like this:

```
1 model = Sequential([
2     # Hidden Layer 1: Use ReLU
3     Dense(units=25, activation='relu'),
4
5     # Hidden Layer 2: Use ReLU
6     Dense(units=15, activation='relu'),
7
8     # Output Layer: Use Sigmoid
9     Dense(units=1, activation='sigmoid')
10 ])
```

- If this were a regression problem where  $y$  could be positive or negative, you would change the last line to `activation='linear'`.